Transferleistung Theorie/Praxis [5

Matrikelnummer:	9102
Freigegebenes Thema:	Latenzreduzierung von RESTful Spring Boot Anwendungen am Beispiel eines Kundenprojektes
Studiengang, Zenturie:	Angewandte Informatik, 18a

Inhaltsverzeichnis

1 Motivation **1**

2 Aktuelle Implementation **1**

3 Versuchsaufbau **2**

4 Optimierungsansätze **3**

4.1 @Query 3

4.2 Mapping 4

4.3 Timestamps als Long anstelle von OffsetDatetime 6

4.4 JSON Umwandlung 6

4.5 Kombination der Verbesserungen 8

5 Aufwand für die Implementierung **8**

6 Fazit **9**

1 Motivation

Wie schnell eine Anwendung auf Benutzereingaben regiert und wie schnell zum Beispiel Webseiten laden, ist wichtig für den Nutzer und lange Ladezeiten können schnell zu Unzufriedenheiten führen. So hat die Ladezeit von Webshops eine klare Auswirkung auf die Kaufbereitschaft der Nutzer [1]. Im Kontext einer Anwendung mit Serverkommunikation, ist daher einerseits wichtig, dass der Frontend Code optimiert ist. An sehr vielen Stellen hat aber auch die Geschwindigkeit, mit der der Server Anfragen beantwortet wichtig. So muss in einigen Fällen erst auf die Antwort einer Anfrage gewartet werden, bevor die Seite initial gerendert wird. An vielen anderen Stellen, zum Beispiel in einer Suche, sind die Ergebnisse von Backend-Anfragen auch der Hauptinhalt der Seite.

Ein häufig eingesetztes Framework für das Backend ist Spring beziehungsweise Spring Boot. In einer Umfrage aus 2020 unter 2000 Befragten für den JVM Ecosystem Report gaben 60% an Spring zu benutzen [2]. In dieser Transferleistung werden an unterschiedlichen Punkten in einer Spring Boot Anwendung Optimierungen durchgeführt und geschaut inwieweit diese die Latenz von Anfragen verbessern.

Zur Optimierung wird ein Teil einer Anwendung betrachtet, mit der über das letzte Jahr Sensordaten gesammelt wurden. Auch Teil der Anwendung ist die Funktion diese Sensordaten auszulesen, um auf einer Website einen Graphen anzuzeigen und einen Export zu ermöglichen. Da über das letzte Jahr mehrere Sensoren ein mal pro Minute Daten geliefert haben, ist die Datenmenge sehr groß geworden und beim Abruf längerer Zeiträume, dauert die Anfrage zuweilen sehr lange ($>$ 30 Sekunden).

Die in dieser Transferleistung erarbeiteten Optimierungen können dann im Anschluss in den Rest der Anwendung aber auch in Referenzimplementierungen übernommen werden, sodass auch neue Anwendungen hiervon profitieren können.

2 Aktuelle Implementation

Wie beschrieben sollen sehr viele Sensordaten in Graphen angezeigt werden. Zur Optimierung werden Anfragen über einen großen Zeitraum bereits vom Frontend aufgeteilt, sodass eine neue Anfrage für jeden Kalendertag im geforderten Zeitraum gesendet wird. Dadurch kann die Parallelisierung des Backends über einerseits mehrere Tomcat Threads, als auch parallele Deployments in Kubernetes ausgenutzt werden, um die Gesamtdauer der Anfragen zu reduzieren.

Dennoch dauert jede Anfrage für jeweils einen Tag noch etwa 5 Sekunden. Aus diesem Grund soll die Antwortzeit von Seiten des Spring Boot Backends verbessert werden. Eine Anfrage hat aktuell den folgenden Ablauf:

- **Ankommende Anfrage:** Die eingehende HTTP Anfrage wird von Spring interpretiert, der entsprechende Controller ermittelt und mit Spring Security die Authentifizierung und Autorisierung überprüft.

- **Parameter interpretieren:** In der aktuellen Version wird für die Anfrage der abgefragte Zeitraum über zwei Datums-/Zeitangaben im ISO 8601 Format angegeben. Diese müssen zur Verwendung zunächst in *OffsetDateTime* Objekte umgewandelt werden.
- **Datenbankabfrage:** Über einen entsprechenden Service wird mithilfe eines JPA Repositories eine Datenbankabfrage mit den gegebenen Parametern gestellt.
- **Mapping:** Im Service werden die Datenbankobjekten auf Data Transfer Objects (DTO) umgewandelt (gemappt), um für den Client überflüssige Daten zu entfernen/anzupassen.
- **Umwandlung in JSON:** Der Controller liefert als Ergebnis eine ResponseEntity mit dem Ergebnisarray als Body. Dieses wird dann automatisch von Spring mithilfe von Jackson in einen JSON String umgewandelt und als Antwort auf die HTTP Anfrage gesendet.

Nachfolgend werden Verbesserungsmöglichkeiten bei den folgenden Aspekten betrachtet: Datenbankquery, Alternativen zum Mapping, Umwandlung zu JSON und die Verwendung von Zahlenwerten statt OffsetDateTime.

3 Versuchsaufbau

Um die Auswirkungen der einzelnen Optimierungen messen zu können, wurde eine neue Spring Boot Anwendung mit einem vergleichbaren Datenbankobjekt, Controller, Service und Repository erstellt. Dadurch können zunächst externe Einflüsse durch andere Services o.ä. vernachlässigt werden. Authentifizierung mit Spring Security wurde explizit weggelassen, da dieses ohne die Verwendung eines Session Cookies sehr viel Latenz hinzufügt und so die Konfiguration des Loadtesters einfacher war. Die Spring Boot Version ist 2.5.3 mit Lombok, Mapstruct und dem PostgreSQL JDBC Driver als relevante Dependencies. Für die Datenbank wurde auf dem gleichen Rechner auf dem auch die Anwendung läuft eine PostgreSQL DB mit der Version 13.3 verwendet. Neben der Testdaten war diese ansonsten leer. Für die Testdaten wurden die Tabellen mit jeweils 1440 Zeilen befüllt. Dies entspricht einem Eintrag für jede Minute des Tages und entspricht damit dem Datenvolumen der zu optimierenden Anwendung.

Für jeden Test wurden mit Apache AB jeweils 1000 direkt aufeinanderfolgende Anfragen gesendet und die Antwortzeiten gemessen. Sowohl Apache AB, Spring Boot und die Datenbank liefen dabei auf dem gleichen Rechner. Daher wird eine mögliche zusätzliche Latenz durch die Netzwerkschichten als vernachlässigbar eingestuft. Als Rechner wurde ein MacBook Pro mit einer 4 Kerne (8 Threads) 2.9 GHz Intel Core i7 CPU, 16 GB RAM und macOS 12.0 verwendet . Alle Tests wurden direkt aufeinanderfolgend durchgeführt für eine maximale Vergleichbarkeit.

4 Optimierungsansätze

4.1 @Query

Zur Datenbankanbindung werden JDBC, JPA und Hibernate verwendet. Für die eigentlichen Zugriffe werden JPA Repositories genutzt. JPA kann automatisch aus den Methodennamen eines Repository-Interfaces die entsprechenden SQL-Queries erstellen[1], was in der aktuellen Implementation ausgiebig genutzt wird, da es schneller zu erstellen und auch einfacher zu warten ist. Ein solches Repository-Interface sieht dann zum Beispiel folgendermaßen aus:

```
@Repository
public interface SampleObjectRepository extends CrudRepository<SampleObject, Long> {
   List<SampleObject> findByLocationProviderIdAndTimestampBetween(
      Long locationProviderId,
      OffsetDateTime startTime,
      OffsetDateTime endTime);
}
```

Lässt man die Anwendung mit dem Java Profiler laufen wird jedoch ersichtlich, dass die Erstellung der SQL-Query lange dauert. Mit dem Debugger wurde herausgefunden, dass an dieser Stelle die Java Reflection API genutzt wird. Die daraus resultierende Query wurde außerdem nicht gespeichert, sodass die zeitaufwändigen Reflection-Aufrufe für jede Anfrage neu durchlaufen werden mussten.

Als Alternative zu den automatisch generierten Queries, kann die Methode auch mit `@Query` annotiert werden. Die gleiche Anfrage könnte dann folgendermaßen aussehen:

```
@Repository
public interface SampleObjectRepository extends CrudRepository<SampleObject, Long> {
   @Query("SELECT s FROM SampleObjectQuery s " +
         "WHERE s.locationProviderId = :locationProviderId " +
         "AND s.timestamp >= :startTime " +
         "AND s.timestamp <= :endTime")
   List<SampleObjectQuery> findByLocationProviderIdAndTimestampBetween(
         @Param("locationProviderId") Long locationProviderId,
         @Param("startTime") OffsetDateTime startTime,
         @Param("endTime") OffsetDateTime endTime);
}
```

Es ist leicht zu sehen, dass diese Form der Querydefinition schwieriger wartbar ist, wenn die Entwicklungsumgebung keinen expliziten Support für Refactoring dieser Annotation liefert. Ebenfalls ist aber leicht erklärbar, wieso diese Form schneller in der Verarbeitung ist, da hier nur die entsprechenden Parameter in den String eingesetzt werden müssen. In Abbildung 1 sind die Anfragedauern im Vergleich zu sehen. Die `@Query`-Annotation verringert in diesem Fall die Anfragedauer um 28% - 31%.

[1] Spring Data JPA Documentation: `https://docs.spring.io/spring-data/jpa/docs/current/reference/html/`

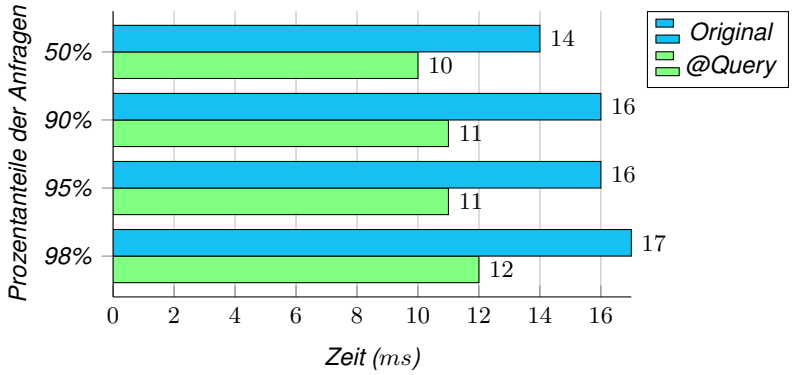

Abbildung 1: Anfragedauer mit Original-Repository im Gegensatz zur `@Query`-Annotation

4.2 Mapping

In vielen Situationen kann es Vorteile bringen, Datenbankobjekte nicht direkt zurückzugeben. So kann durch das Weglassen bestimmter Felder die Datenrate verringert werden und die interne Struktur der Daten weniger transparent für Anwender aber auch Angreifer gemacht werden. Durch die Umbenennung von Exemplarvariablen kann zudem die Rückwärtskompatibilität für APIs gewährleistet werden. Zu diesem Zweck werden Data Transfer Objects (DTOs) verwendet.

In diesem konkreten Fall können ohne Informationsverlust für den Client zwei interne Ids weggelassen werden. Dazu wird die Library MapStruct verwendet. Die erlaubt ein Interface mit entsprechenden Methodennamen zu definieren, aus dem automatisch eine Implementation generiert wird. Mithilfe der `@Mapping`-Annotation können zusätzlich Exemplarvariables transformiert oder auf anderes benannte Variables übertragen werden.

Die Ziele des Mappings können in vielen Fällen allerdings mit weniger rechenaufwändigen Operationen und Wegen erreicht werden. Für das Mapping muss nämlich über alle Objekte iteriert werden und jeweils ein neues Objekt initialisiert werden. Gerade wenn längere Listen, wie im konkreten Fall vorliegend, umgewandelt werden sollen, dauert dies sehr lange. Im folgenden werden zwei alternative Wege gezeigt.

4.2.1 Jackson Annotationen

Jackson wird in Spring Boot standardmäßig zur Umwandlung von Objekten in eine JSON Repräsentation verwendet. Als Unterstützung stellt Jackson einige Annotationen zur Anpassung des Ergebnisses zur verfügung. Mit `@JsonIgnore` kann so eine Variable weggelassen und mit `@JsonProperty` kann der Name im JSON verändert werden[2]. Diese Annotationen können dann direkt an den Variablen der Datenbank Entity verwendet werden. Das hat den Nachteil, dass die gleichen Annotationen für diese Entity Anwendungsweit verwendet werden müssen, reduziert aber auch die Menge an sonst benötigtem, zusätzlichen Code.

[2] Jackson Annotations: `https://github.com/FasterXML/jackson-annotations/wiki/Jackson-Annotations`

Die hiermit erreichte Verbesserung ist in Abbildung 2 zu sehen. Auch durch diese Maßnahme wurde die Anfragedauer deutlich reduziert im Gegensatz zum Original-Code. Hier wurden erneut ca. 30% eingespart.

4.2.2 Direkt DTO laden

Wenn das Ziel des Mappings nur ist Felder zu entfernen oder anders zu benennen, können mit JPA Repositories auch ausschließlich die benötigten Spalten direkt als DTOs geladen werden. Dadurch wird dann neben der geringeren Datenübertragung zum Client auch die Datenmenge zwischen Spring Boot Anwendung und Datenbank reduziert. Ein Nachteil ist aber, dass hier mehr Aufwand im Repository von Nöten ist.

```
1  @Repository
2  public interface SampleObjectRepository extends CrudRepository<SampleObject, Long> {
3     @Query("SELECT new package.name.SampleObjectDTO(s.timestamp, s.longitude, s.latitude, " +
4           "s.pressure, s.temperature, s.speed) FROM SampleObject s " +
5           "WHERE s.locationProviderId = :locationProviderId " +
6           "AND s.timestamp >= :startTime " +
7           "AND s.timestamp <= :endTime")
8     List<SampleObjectDTO> findByLocationProviderIdAndTimestampBetween(
9        Long locationProviderId, OffsetDateTime startTime, OffsetDateTime endTime);
10 }
```

Dadurch, dass für diese Methode ebenfalls die `@Query`-Annotation verwendet wird, profitiert sie von dieser Verbesserung automatisch. Durch das Entfernen des Mappings wird die Latenz um weitere 40% gegenüber der reinen `@Query`-Verbesserung reduziert.

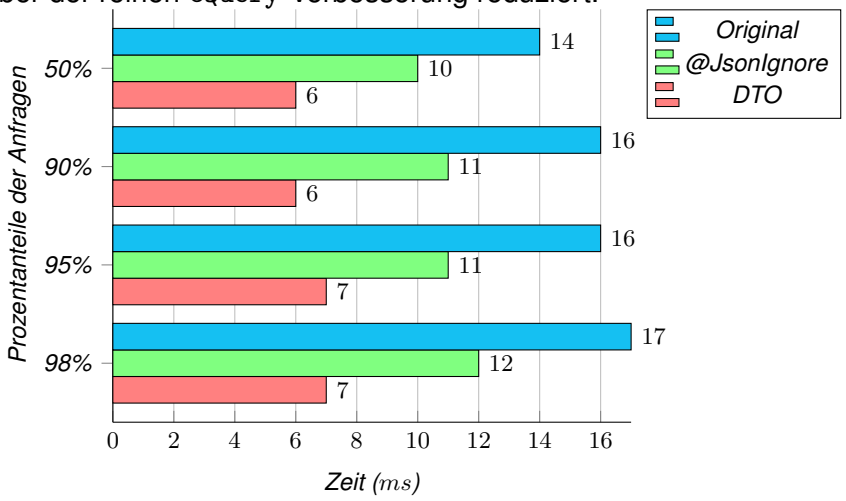

Abbildung 2: Anfragedauer mit Mapping im Vergleich zu `@JsonIgnore` und DTOs direkt zu laden

4.3 Timestamps als Long anstelle von OffsetDatetime

Da die Datenpunkte im betrachteten Beispiel eine Zeitreihe darstellen, enthalten die Objekte jeweils einen Timestamp in der Form eines `OffsetDatetime`. Dadurch wird auch automatisch der Datentyp `Timestamp` in der Datenbank verwendet. Diese Nutzung sorgt allerdings für viele Umwandlungen der Zeitpunkte:

- Die Anfrage an das Backend umfasst einen Start- und Endzeitpunkt, um den abgefragten Zeitraum entsprechend einzugrenzen. In der HTTP Anfrage sind diese in Form eines ISO-8601 Strings. Für die Weiterverarbeitung innerhalb der Spring-Boot Anwendung werden diese zunächst in `OffsetDatetime`-Objekte umgewandelt.
- Bei der Anfrage an die Datenbank wird die `OffsetDatetime` wieder in einen String umgewandelt, um SQL konform zu sein.
- Die Datenbank muss diesen String wieder analysieren, um den Zeitpunkt mit den Zeitreiheobjekten vergleichen zu können.
- Die Timestamps im Ergebnis der Datenbankabfrage werden wieder in `OffsetDatetime`-Objekte umgewandelt.
- Für die Antwort an den Client in Form von JSON muss erneut der Timestamp als String eingefügt werden.

Es finden hier also sehr viele Umwandlungen statt. Gerade die Umwandlung von einem String in ein `OffsetDatetime` ist aber sehr aufwändig. Dafür muss der String an den richtigen Stellen geteilt werden, die einzelnen UTF-8 Zeichen als Zahlen interpretiert und anschließend als ein einzelner Timestamp zusammengeführt werden. Stattdessen könnte die `OffsetDatetime` auch an allen Stellen durch einen Long ersetzt werden, in dem der Zeitpunkt z.B. als Sekunden seit dem 01.01.1980 gespeichert wird. Diese Form der Speicherung ist ebenfalls weit verbreitet, man verliert jedoch ggf. Zeitzoneninformationen. Sind diese aber nicht von Nöten, weil alle Timestamps sowieso in der gleichen Zeitzone liegen oder an anderer Stelle umgerechnet werden können, so kann die Nutzung von Longs die Latenz stark minimieren, weil die Umwandlung nur noch von String zu Long stattfinden muss und keine aufwändige Analyse eines ausgeschriebenen Datums notwendig ist. In Abbildung 3 ist diese Latenzverbesserung in Höhe von etwa 55-60% dargestellt.

4.4 JSON Umwandlung

Wenn ein REST-Controller auf die Anfrage eines Clients ein Objekt zurückgibt, so wird dieses von Spring standardmäßig in JSON umgewandelt. Diese Umwandlung nimmt, je nach Anfrage, einen relativ großen Anteil der Anfragedauer ein[3]. Für diese Umwandlung nutzt Spring Boot als Standard

[3] Bei der getesteten Anfrage betrug der Anteil ca. 30-40%. Dies kann relativ einfach mit dem Java Profiler überprüft werden.

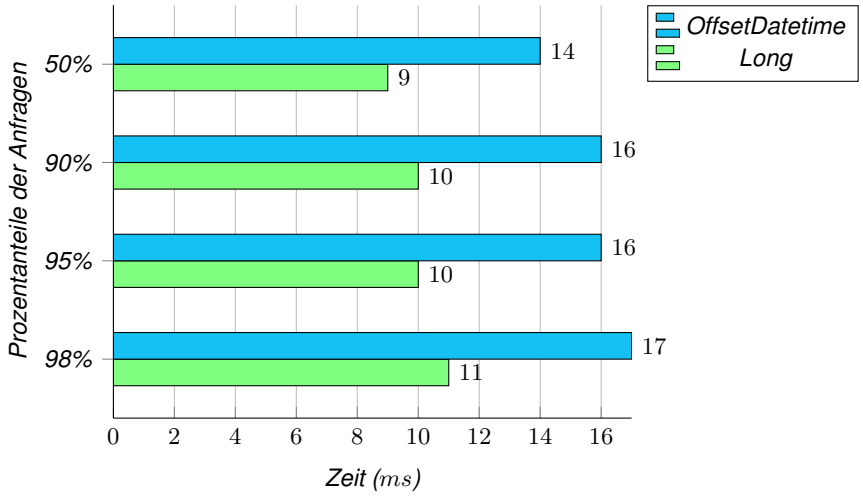

Abbildung 3: Anfragedauer Long anstelle von `OffsetDatetime`

Jackson. Wird allerdings die Jackson Streaming API direkt angesprochen, wird aus unbekannten Gründen die Anfragedauer aber deutlich reduziert.

```
@GetMapping(value = "/{locationProviderId}")
public ResponseEntity<StreamingResponseBody> getAllEntries(...) {
   StreamingResponseBody stream = out -> {
      objectWriter.writeValue(out,
            sampleObjectService.getEntriesForTimeRange(
                  locationProviderId,
                  OffsetDateTime.parse(startTime),
                  OffsetDateTime.parse(endTime))
      );
      out.flush();
   };
   return ResponseEntity.ok()
         .contentType(MediaType.APPLICATION_JSON)
         .body(stream);
}
```

Einfach durch die Nutzung dieses Codes wird die Latenz um 30 bis 45 Prozent verbessert. Dies ist auch in Abbildung 4 zu sehen. Mit einer generischen Implementation ließe sich dieser Code auch relativ einfach auslagernd, um Dopplungen zu vermeiden.

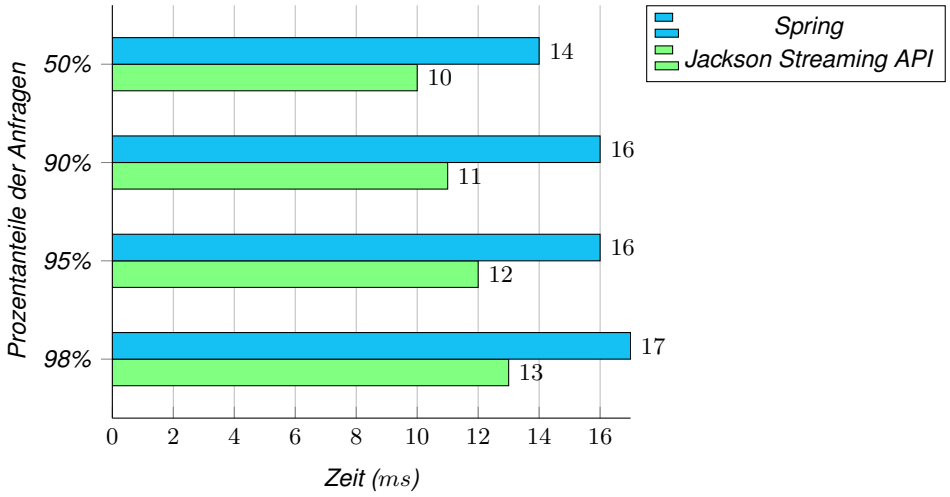

Abbildung 4: Anfragedauer Jackson Streaming API vs Spring-Boot Standard

4.5 Kombination der Verbesserungen

Alle oben genannten Verbesserungen können wie in den obigen Versuchen einzeln implementiert werden. Genauso kann man sie aber auch miteinander kombinieren. Im Falle des Mappings wurde hierfür die Variante mit DTO verwendet. Wird dies gemacht, so kann die Latenz um einen Faktor von 3,2 bis 3,5 mal verbessert werden. Diese Verbesserung ist auch in Abbildung 5 dargestellt.

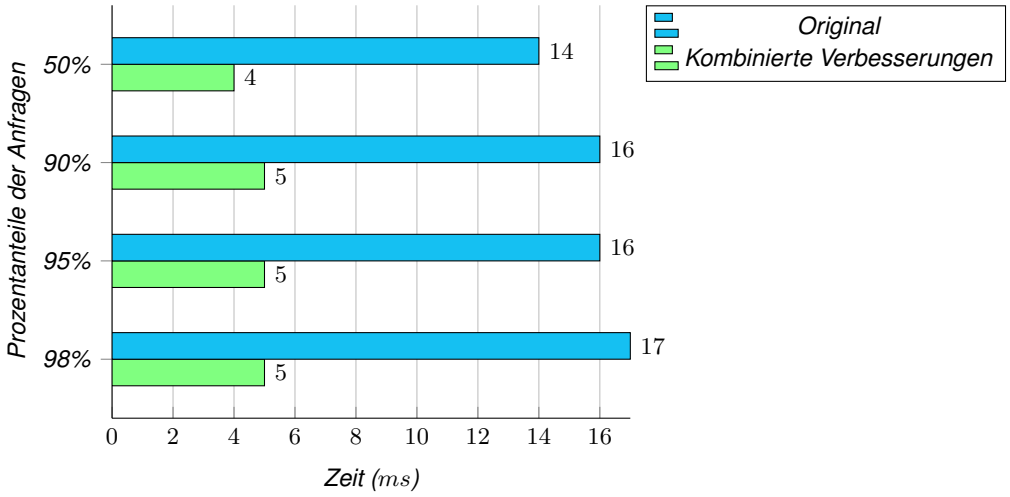

Abbildung 5: Anfragedauer Jackson Streaming API vs Spring-Boot Standard

5 Aufwand für die Implementierung

Generell muss für jede Optimierung betrachtet werden, ob sich der Aufwand dafür lohnt, gerade, wenn eine minimale Anfragedauer zwar erstrebenswert aber nicht notwendig ist. Im folgenden wird für jede Optimierung kurz der Aufwand geschildert, der für eine Implementation notwendig ist, um

daraus ableiten zu können, wann sie vorgenommen werden sollte:

@Query

Die Verwendung von `@Query` ist eine der aufwändigsten Optimierungen. Hierfür muss manuell die SQL-Query geschrieben und auf Optimierungen innerhalb der Query geachtet werden. Außerdem hat nicht jede IDE die Fähigkeiten Variable- und Klassennamen innerhalb der Query automatisch umzubenennen, was die Wartbarkeit deutlich reduziert.

Mapping

Die Nutzung von Mappern und Mapstruct sollte vermieden werden, wenn möglich. Mit der `@JSONIgnore`-Annotation lässt sich in einigen Fällen mit deutlich weniger Aufwand eine um einiges bessere Performance erreichen. Für andere Fälle sollte eher ein DTO direkt über die `@Query`-Annotation initialisiert werden. Diese Variante ist ca. 60% schneller.

Timestamps als Long

Diese Änderung sorgt dafür, dass die API angepasst werden muss, wenn man sie vollständig umsetzen möchte. Bestehende Schnittstellen sollten nur dann angepasst werden, wenn dies wirklich notwendig ist. Wenn neue Schnittstellen und Objekte erstellt werden, sollte aber abgewägt werden, ob Zeitzoneninformationen dringend benötigt werden oder ob eine 55-60% bessere Latenz nicht überwiegt.

JSON Umwandlung

Diese Optimierung kann relativ einfach generisch umgesetzt werden und verbessert damit die Anfragedauern der gesamten Anwendung ohne langfristigen Mehraufwand. Auch in bestehende Anwendungen kann sie einfach implementiert werden und konnte die Latenz im Beispiel um 30-45 Prozent verbessern.

6 Fazit

Mit vielen recht einfachen Verbesserungen kann insgesamt die Latenz deutlich verbessert werden. Von den genannten Möglichkeiten würde dabei ausschließlich die Verwendung von Long anstelle von `OffsetDatetime` für Timestamps eine Änderung an der REST-API benötigen. Alle anderen Änderungen können gemacht werden, ohne dass Clients angepasst werden müssten.

Viele Schnittstellen würden vermutlich nicht von den Verbesserungen profitieren. Gerade bei POST-, UPDATE- und DELETE-Anfragen, die normalerweise innerhalb weniger Millisekunden beantwortet werden können, wäre selbst eine Verbesserung um den Faktor 3,5, wie er hier erreicht werden konnte, für den Nutzer kaum merkbar.

Bei einigen Anfragen ist der Nutzen aber deutlich größer. Anfragen die vor dem ersten Render einer Website durchgeführt werden müssen und zu lange brauchen, können sich negativ auf sowohl

Suchmaschienen-Rankings als auch auf die Kaufbereitschaft von Kunden auswirken [1].

Auch bei Anfragen, die sehr lange dauern, sind Optimierungen sinnvoll. Dauert eine Anfrage nur fünf statt zehn Sekunden, so ist dieser Effekt deutlich merkbarer als 40ms im Gegensatz zu 80ms. Dies kann die Nutzererfahrung deutlich verbessern.

Zur Nachstellung der Ergebnisse und als Beispielimplementation zum Nachbauen in anderen Anwendungen ist der gesamte Code inklusive des Testscriptes auf GitHub unter `https://github.com/jonas-peeters/spring-performance` zu finden.

Literatur

[1] W. Stadnik und Z. Nowak, „The Impact of Web Pages' Load Time on the Conversion Rate of an E-Commerce Platform," in *Information Systems Architecture and Technology: Proceedings of 38th International Conference on Information Systems Architecture and Technology – ISAT 2017*, L. Borzemski, J. Świątek und Z. Wilimowska, Hrsg., Cham: Springer International Publishing, 2018, S. 336–345, ISBN: 978-3-319-67220-5.

[2] Synk, „JVM Ecosystem Report 2020," 2020. Adresse: `https://snyk.io/wp-content/uploads/jvm_2020.pdf`.